Reading a plan: the short version

The rules with their reasons, the instruments, and two real plans answered the way the exam grades

Advanced Data Management

2026-11-01

0 What a plan is, in five sentences

The planner turns a query into a tree of operators and `EXPLAIN` prints that tree, one node per line, children indented under their parent. Each node is an iterator: the parent asks it for the next row, it asks its own children, and rows flow upwards one at a time, so most of the tree *streams*; a few nodes (Sort, Hash build, HashAggregate) must see all their input before emitting anything, and these *breakers* are where waiting and memory happen. `EXPLAIN` alone shows what the planner *believed*: costs and row counts computed from statistics before anything ran. `EXPLAIN ANALYZE` runs the query and prints, next to each belief, what actually happened; `BUFFERS` adds how many pages each node asked for and how many were already in memory. Reading a plan is comparing belief with reality node by node, from the leaves up, and finding the lowest place where they part.

1 The lines and what they mean

Concept: Node line

`(cost=start..total rows=est width=w)`: the planner's *belief*, in abstract units (not ms). `start` = work before the first row leaves the node (for a breaker: its whole input). `rows` = estimated rows *per execution*.
`(actual time=first..last rows=act loops=n)`: measured, *per loop*. A node on the inner side of a Nested Loop runs once per outer row, so: total rows = `act` × `loops`; time = `last` × `loops`. `first..last` is first row and last row, not a duration.
Why per loop: the inner side of a Nested Loop is re-executed for every outer row, and PostgreSQL averages its figures over those executions; a line that says `rows=1 loops=120000` produced 120,000 rows and `time=0.020..0.024` is 24 μs per execution, 2.9 s in total. Forgetting to multiply is the most common misreading of a plan.
Why cost units are not time: they are calibrated so that one sequential page read costs 1; random pages cost 4, a row 0.01, an index entry 0.005, a comparison 0.0025. The planner compares plans by these numbers; you convert to seconds only through pages and device latency.

Concept: Sub-lines

- `Filter` + `Rows Removed by Filter`: the predicate was tested on each row *after* reading it; the removed count is wasted work.
- `Index Cond`: answered inside the index, no row read for it.

- `Recheck Cond` + `Heap Blocks: exact=N`: a bitmap scan; N is the number of heap pages it visited.
- `Heap Fetches`: index-only scan; 0 means no heap page at all.
- `Hash Cond` / `Merge Cond` / `Join Filter`: the join condition (the Join Filter is applied after pairing).
- `Buckets ... Batches ... Memory Usage`: the hash build; `Batches` > 1 means it spilled to disk.
- `Sort Key`, `Sort Method: quicksort Memory` (fit) / `external merge Disk` (spilled) / `top-N heapsort` (ORDER BY + LIMIT).
- `Group Key`: the GROUP BY columns.
- `Buffers: shared hit=H read=R`: pages found in `shared_buffers` / not found there (fetched from the OS cache *or* disk; `read` $\neq$ disk). Per node, children included. `temp read/written` = a spill.

Why Filter and Index Cond differ: an `Index Cond` is used to *navigate* the index, so rows that fail it are never touched; a `Filter` is applied to rows already read, so every removed row cost a read. A scan with `Rows Removed by Filter: 2880000` read 2.88 million rows for nothing, which is the first thing an index on that predicate could change (if the matches are not scattered over every page anyway).

Why Heap Blocks matters: a bitmap scan visits each matching heap page once, in physical order; `Heap Blocks` is that count, and comparing it with the table's page count says whether the index saved anything. 27,922 of 28,038 means the index found the rows but the scan still read the table.

Why Batches and Disk matter: they say a breaker did not fit in `work_mem` and went through disk; the figure next to `Disk` is roughly the memory that would have avoided it, to be granted with `SET LOCAL work_mem` in that transaction only, because `work_mem` is per sort or hash and per connection.

Concept: The tree

Indentation gives the children. For a join, the first child is the **outer** (nested loop) or **probe** (hash join) side, the second the **inner** or **build** side (the one under `Hash`). Read bottom-up; execution starts at the leftmost leaf of the deepest breaker.

Blocking (consumes all input before its first output): Sort, Hash (build), HashAggregate, Materialize, Bitmap Index Scan. **Streaming**: Seq Scan, Index Scan, Bitmap Heap Scan, Filter, Limit, Nested Loop, Merge Join, hash probe, GroupAggregate on sorted input. A LIMIT above a breaker saves nothing below it.

Reconstruct the SQL: Scan on `t a` = FROM; `Hash/Merge Cond` = ON; `Filter`/`Index Cond` = WHERE; `Group Key` = GROUP BY; `Sort Key` = ORDER BY; `Limit` = LIMIT.

Technique: The five moves, in the order you write them

1. **Leaves**: for each table, scan type, filter, estimated vs actual rows.
2. **Up**: join algorithm and order, aggregate, sort.
3. **Cause**: the *lowest* node with a large q-error ($\max(\text{est}/\text{act}, \text{act}/\text{est}) \geq 10$); everything above inherits it. An under-estimate is the dangerous direction: it buys per-row plans (nested loop, index lookups) for a big input.
4. **Resources**: `Buffers` per node (the `read` count, not a ratio), `Sort Method`, `Batches`, `loops` on the inner side, `Heap Blocks`. Seconds $\approx$ `read` $\times$ random price.
5. **Decide**: wrong plan from a wrong estimate $\rightarrow$ statistics fix; right plan but slow $\rightarrow$ memory, index, cache; name the command.

Concept: Where an estimate comes from, and the three fixes

Scan `rows=` = (MCV frequency, else $1/n_{\text{distinct}}$, else histogram share) $\times N$; each extra predicate multiplies by its own selectivity (independence); join rows by the join formula.
Stale (`n_live_tup` $\neq$ `reltuples`, old `last_analyze`) $\rightarrow$ `ANALYZE t;`
Correlated columns (each frequency right, the product wrong; ANALYZE does not help) $\rightarrow$ `CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t;`
Skew (a rare value estimated at $1/d$) $\rightarrow$ `ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000; ANALYZE t;`

Concept: What each fix changes

Extended statistics: estimate right $\rightarrow$ join algorithm and order change. **Composite index on the filter**: Filter moves into Index Cond; pages unchanged if rows are scattered. **CLUSTER**: correlation $\rightarrow$ 1, Heap Blocks collapse, the join untouched; a rewrite under lock. **Covering index**: index-only, Heap Fetches 0. **work_mem**: Disk/Batches disappear. **Hash join instead of nested loop**: build on the smaller *filtered* input (check rows $\times$ width against work_mem), probe with a Seq Scan: sequential pages instead of loops of random ones.

2 The instruments around the plan

Concept: What to run, and what each answer tells you

- `EXPLAIN q`: the plan and estimates only; nothing runs; safe on production; shows the planner's belief.
- `EXPLAIN ANALYZE q`: runs it; adds actual rows, times, loops, `Sort Method`, `Batches`, `Execution Time`. For DML wrap it: `BEGIN; EXPLAIN ANALYZE DELETE ...; ROLLBACK;`
- `EXPLAIN (ANALYZE, BUFFERS) q`: adds hit/read/temp per node, the only view of the buffer pool's behaviour for this query.
- `SELECT attname, null_frac, n_distinct, most_common_vals, most_common_freqs, histogram_bounds, correlation FROM pg_stats WHERE tablename = 't';` the numbers every estimate was computed from. `n_distinct` negative = a fraction of the rows; `correlation` near ± 1 = matches contiguous, near 0 = scattered.
- `SELECT reltuples, relpages FROM pg_class WHERE relname = 't';` the size the planner believes; `pg_relation_size('t') / 8192` the real pages; a gap means dead tuples or stale statistics.
- `SELECT n_live_tup, n_dead_tup, last_analyze, last_autoanalyze FROM pg_stat_user_tables WHERE relname = 't';` is the table stale?
- `SELECT heap_blks_hit, heap_blks_read FROM pg_statio_user_tables WHERE relname = 't';` the table's hit ratio over time.
- After any fix: `ANALYZE t;` then re-run `EXPLAIN (ANALYZE, BUFFERS)` and compare the node you changed.

Concept: How to read cold versus warm

Run the query twice with `BUFFERS`. If the second run has `read=0` and the same time, the query is CPU-bound: all pages were already in memory the first time and the plan is the problem. If the second run is much faster, the first was paying `read`s: the plan may be fine and the cache was cold, or the working set does not fit. A `read` served by the OS cache costs a few microseconds, by an SSD about 100, by a disk 10 ms; when the question gives a measured time, infer the effective cost per `read` from it (3 s / 50,000 = 60 μs) rather than assuming the device figure.

3 Plan A: a nested loop that should have been a hash join

`bookly`: `orders` 3,000,000 rows, 28,038 pages, 107 rows/page; `customers` 300,000 rows, 2,500 pages, `customers_pkey` 3 levels. `pg_stats`: `status` MCV `returned` 0.05; `country` MCV `FR` 0.20, both fresh. A random read costs about 60 μs here (part OS cache, part SSD). `work_mem` 4 MB.

```
SELECT c.name, count(*) AS n, sum(o.amount) AS refunded
FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE o.status = 'returned' AND o.country = 'FR'
GROUP BY c.name ORDER BY refunded DESC LIMIT 20;
```

```
Limit  (cost=115982.1..115982.2 rows=20)
       (actual time=3480.2..3480.3 rows=20 loops=1)
  Buffers: shared hit=458038 read=50000
  ->  Sort  (cost=115982.1..116057.1 rows=30000)
            (actual time=3480.1..3480.2 rows=20 loops=1)
        Sort Key: (sum(o.amount)) DESC
        Sort Method: top-N heapsort  Memory: 27kB
        ->  HashAggregate  (cost=114632.1..114932.1 rows=30000)
                  (actual time=3440.6..3462.3 rows=98210 loops=1)
              Group Key: c.name
              ->  Nested Loop  (cost=0.42..114482.1 rows=30000)
                        (actual time=0.3..3390.8 rows=120000 loops=1)
                    ->  Seq Scan on orders o
                          (cost=0.00..65538.0 rows=30000)
                          (actual time=0.1..410.5 rows=120000 loops=1)
                          Filter: ((status = 'returned') AND (country = 'FR'))
                          Rows Removed by Filter: 2880000
                          Buffers: shared hit=28038
                    ->  Index Scan using customers_pkey on customers c
                          (cost=0.42..1.63 rows=1)
                          (actual time=0.020..0.024 rows=1 loops=120000)
                          Index Cond: (id = o.customer_id)
                          Buffers: shared hit=430000 read=50000
Execution Time: 3482.0 ms
```

Worked example: Q1. Tree, blocking or streaming, why LIMIT does not help

Limit ← Sort ← HashAggregate ← Nested Loop {outer: Seq Scan `orders`; inner: Index Scan `customers`, once per outer row}. Seq Scan: streaming (filters row by row). Index Scan: streaming (one lookup per outer row). Nested Loop: streaming (emits each pair as the inner returns it). HashAggregate: **blocking** (a customer's sum is final only after its last order). Sort: **blocking** (the largest sum is known only when all groups are in). Limit: streaming.

Read it off the times: the join ends at 3,390.8 ms, the HashAggregate's first output is at 3,440.6 ms, the Sort's at 3,480.1 ms, the Limit's 20 rows at 3,480.3 ms. The LIMIT saved only the sort's output phase (`top-N heapsort`, 27 kB); 3.4 s were spent below two breakers that cannot stop early.

Worked example: Q2. Where do the estimates come from, q-error, the failure, the command

`rows=30000` on the Seq Scan: $0.05 \times 0.20 = 0.01$, times 3,000,000: two MCV frequencies multiplied under independence. Actual 120,000: q-error = 4, an *under*-estimate. French orders are returned four times more often than average: `status` and `country` are correlated. `ANALYZE` does not help (both single frequencies are already right). Fix:

```
CREATE STATISTICS s_ret (dependencies) ON status, country FROM orders;  ANALYZE orders;
```

New estimate $\approx$ 120,000, and at that count the planner abandons the nested loop (Q4).

Worked example: Q3. Where the time goes, and the Buffers line

Own times: Seq Scan 410 ms; inner lookups $0.024 \times 120{,}000 = 2{,}880$ ms; HashAggregate 72 ms; Sort 18 ms. The inner lookups own 83 %.
`hit=430000 read=50000`: 120,000 lookups × (3 index pages + 1 heap page) = 480,000. The index pages are the same hot pages every time (hits); the heap page differs per customer and 50,000 of those fetches missed. $50{,}000 \times 60\,\mu\text{s} = 3.0$ s ≈ the 2.9 s measured: the time is the random heap reads of `customers`.
Hit ratio 458,038/508,038 = 90 %: useless here; the cost is the 50,000 `reads` on one node. A bitmap scan on a `status` index would show `Heap Blocks` $= 28{,}038 \times (1 - 0.95^{107}) = 27{,}922$, the whole table: no gain.

Worked example: Q4. The plan the planner should have chosen

With 30,000 outer rows the lookups were priced $30{,}000 \times 1.63 \approx 49{,}000$; at the true 120,000 they are 195,600, so the plan as run is worth $65{,}538 + 195{,}600 \approx 261{,}000$ units (3.5 s). Hash join: build on the smaller *filtered* input, the 120,000 `orders` rows × 80 B = 9.6 MB > 4 MB (`Batches: 4`; `SET LOCAL work_mem = '16MB'` for one batch); probe with a Seq Scan of `customers`, $2{,}500 + 300{,}000 \times 0.01 = 5{,}500$. Total ≈ $65{,}538 + 5{,}500 + 0.0025 \times 420{,}000 \approx 72{,}000$ units, 3.6 times less; in time, 2,500 sequential pages (40 ms) instead of 50,000 random reads: about 0.5 s instead of 3.5 s.

Worked example: Q5. Four proposals, what each changes, which first

(i) `CREATE INDEX ON orders (status, country)`: Seq Scan becomes a Bitmap Heap Scan, `Heap Blocks` ≈ 18,400 (1 % of rows, scattered), estimate still 30,000, the 120,000 lookups remain: 3.3 s. (ii) `CLUSTER orders USING` that index: matches become $\lceil 120{,}000/107 \rceil = 1{,}122$ contiguous pages, the scan drops to ms, the join unchanged: 3.0 s, plus a 219 MB rewrite under lock that decays. (iii) extended statistics: estimate 120,000 → hash join: 0.5 s. (iv) covering index `(status, country) INCLUDE (customer_id, amount)`: index-only scan of 343 leaf pages, `Heap Fetches: 0`, the join unchanged: 3.0 s.
First (iii): two commands, no rewrite, it fixes the node that owns the time. Then (iv) if the remaining 0.4 s scan matters. (i) and (ii) spend effort on the 12 % of the time the scan owns.

Warning / Gotcha: The traps

Reading inner-side `rows=1` as one row (it is per loop). Reading `actual time=a..b` as a duration. "`read` means disk". Judging by the global hit ratio. Fixing the scan when the join owns the time. Blaming the top node. Building the hash on the larger unfiltered table. Adding an index for a 4× estimate error.

4 Plan B: a hash join, a bitmap scan and a spilled sort

Same database. `orders_ts_idx` on `order_ts` (3 levels, correlation 0.98). `work_mem` 4 MB.

```
Sort  (cost=98412.5..99237.5 rows=330000)
      (actual time=2210.4..2410.9 rows=329880 loops=1)
  Sort Key: c.country, o.order_ts
  Sort Method: external merge  Disk: 28960kB
  Buffers: shared hit=3301 read=1160, temp read=3620 written=3640
  ->  Hash Join  (cost=6450.0..61210.0 rows=330000)
                 (actual time=48.2..1620.3 rows=329880 loops=1)
        Hash Cond: (o.customer_id = c.id)
        ->  Bitmap Heap Scan on orders o  (cost=3530.0..41880.0 rows=330000)
              (actual time=21.5..880.6 rows=329880 loops=1)
              Recheck Cond: (order_ts >= '2026-08-01')
              Filter: (amount > 100)
              Rows Removed by Filter: 288120
              Heap Blocks: exact=5780
              Buffers: shared hit=801 read=1160
              ->  Bitmap Index Scan on orders_ts_idx
                    (cost=0.00..3447.5 rows=618000)
                    (actual time=19.8..19.8 rows=618000 loops=1)
                    Index Cond: (order_ts >= '2026-08-01')
        ->  Hash  (cost=2500.0..2500.0 rows=300000)
                  (actual time=25.9..25.9 rows=300000 loops=1)
              Buckets: 524288  Batches: 1  Memory Usage: 18432kB
              ->  Seq Scan on customers c  (cost=0.00..2500.0 rows=300000)
                    (actual time=0.0..18.1 rows=300000 loops=1)
Execution Time: 2455.0 ms
```

Worked example: Q1. The SQL, the tree, who blocks

```
SELECT ... FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.order_ts >= '2026-08-01' AND o.amount > 100
ORDER BY c.country, o.order_ts;
```

Sort ← Hash Join {probe: Bitmap Heap Scan `orders` ← Bitmap Index Scan; build: Hash ← Seq Scan `customers`}. Blocking: Bitmap Index Scan (the bitmap must be complete), Hash (the build must be complete before the first probe), Sort. Streaming: Bitmap Heap Scan, the probe side of the join, Seq Scan. Execution starts at the Seq Scan of `customers` (the build is under the deepest breaker on the right), then the bitmap is built, then rows stream through the probe into the Sort, which emits nothing until 2,210 ms.

Worked example: Q2. Are the estimates right?

Bitmap Index Scan: 618,000 estimated, 618,000 actual ($0.206 \times 3{,}000{,}000$: the range covers 150 of 730 days). Bitmap Heap Scan after `amount > 100`: histogram says $(205 - 100)/200 \approx 0.53$, $618{,}000 \times 0.53 \approx 330{,}000$; actual 329,880, q-error 1.0. Hash Join: 330,000 estimated, 329,880 actual. Every node is within 1 %: the statistics are fine and the plan is the right plan; the time is a resource problem, not a planning problem. (Had the Sort been fed 10× more rows than estimated, that would have been the cause instead.)

Worked example: Q3. Where the time goes

Own times: Seq Scan `customers` 18 ms; Hash build 8 ms; Bitmap Index Scan 20 ms; Bitmap Heap Scan 861 ms; Hash Join probe $1{,}620 - 881 - 26 \approx 713$ ms; Sort $2{,}411 - 1{,}620 = 791$ ms. Two owners: the heap scan and the sort.

The heap scan: `Heap Blocks: exact=5780`: 618,000 matching rows at correlation 0.98 sit on $\lceil 618{,}000/107 \rceil = 5{,}776$ contiguous pages, exactly what the bitmap visited; of these, `read=1160` were

not in the pool. 288,120 rows were read and discarded by the `Filter` on `amount`: the index narrowed by time only.
The sort: `external merge Disk: 28960kB`: 330,000 rows did not fit in 4 MB; `temp read=3620 written=3640` is the spill traffic (29 MB written and read back). **The hash**: `Batches: 1, Memory Usage: 18432kB`: the 300,000 customers fit in one batch (the hash build is allowed its own `work_mem`).

Worked example: Q4. The fixes, and what each changes

(i) `SET LOCAL work_mem = '64MB'` for this query: the Sort becomes `quicksort Memory: 40MB`, `temp` disappears, about 0.5 s saved; nothing else changes. (ii) An index on `(order_ts) INCLUDE (amount, customer_id)`: the heap scan becomes an index-only scan, `Heap Fetches: 0`, no 5,780 heap pages and no `read`s; the filter on `amount` still discards 288,120 index entries. (iii) A composite `(order_ts, amount)`: `amount > 100` moves into the `Index Cond` only as a range on the second column, which a B-tree cannot use to skip within a time range: little gain. (iv) A sort-merge join instead of hash: no, the output order is `(country, order_ts)`, not the join key; the sort is needed anyway. **First (i)**: one line, no object created, it fixes the spill; then (ii) if the 0.9 s scan matters. The plan shape stays: this is a right plan made cheaper, not a wrong plan replaced.

Worked example: Q5. The Buffers line, top to bottom

Top: `shared hit=3301 read=1160`: 2,500 pages of `customers` + 801 hits + 1,160 reads on `orders`' 5,780 heap pages + a few index pages; the join and the sort themselves read no new pages. Hit ratio $3{,}301/4{,}461 = 74\,\%$, and here the `read`s are 1,160 *sequential-ish* pages (contiguous blocks), so they cost tens of ms, not seconds: the same 74 % that would be alarming for random lookups is harmless here. The `temp` figures belong to the sort, not to the pool: raising `shared_buffers` would not touch them; `work_mem` does.

Warning / Gotcha: What plan B teaches that plan A does not

A plan can be the right plan and still slow: when every estimate is within a few percent, stop looking for a statistics fix and look at resources (`Disk`, `Batches`, `read`). `Heap Blocks` close to k/r means contiguous matches, the index did its job. A `read` count is cheap or expensive depending on whether the pages are contiguous. `temp` is memory, not cache.